

Land Use Data Analysis Toolbox for ArcGIS

A modular suite of roughly twenty linked ArcGIS Python Toolboxes and standalone scripts that together turn raw statewide parcel, soil, flood, water, employment, and housing data into a governed, auditable land use inventory

The Problem

A statewide land use inventory needs to combine data from a lot of independent, messy sources, county parcel records, USDA soil surveys, FEMA flood hazard maps, EPA water service boundaries, state employment micro-data, and Census housing statistics, into one consistent, defensible dataset. Doing that as a single monolithic process, or as a set of disconnected one-off scripts, runs into problems that get worse the longer the system lives.

- Every one of these source datasets has its own quirks, changing publication dates, inconsistent county-level coding, unreliable network access, geocoding of varying precision, and each needs its own tailored handling rather than a one-size-fits-all import step.
- Business rules like land use classification codes, county data quality tiers, and validation thresholds change over time as the project matures. A system where those rules live buried in code becomes harder to maintain and easier to get wrong with every passing year.
- At statewide scale, the most expensive operations, spatial joins and multi-layer overlays, can take hours if built naively, and a slow pipeline stage discourages the kind of frequent re-running that catches problems early.
- Data quality varies a lot by source and by county. Without built-in auditing and approval gates, bad data from one weak link can flow silently through the whole system and show up as a wrong number in a final deliverable with no trace of where it went wrong.
- A pipeline this size needs to be debuggable months after it was written. Without consistent logging, config-driven rules, and validation at every stage, tracking down why a specific number looks wrong becomes a full investigation instead of a quick log check.

My Approach

I designed and built this as a modular toolbox, roughly twenty ArcGIS Python Toolboxes and standalone Python scripts, each one owning a single, clearly scoped stage of the overall pipeline, so the system can run end to end or be re-run at just the stage that needs attention.

A staged architecture, not a monolith. The toolbox is organized into clear phases: configuration sync and data acquisition, environmental and terrain data preparation (soil-derived slope, floodplain, water service boundaries), parcel cleaning and classification, a

multi-layer terrain overlay that combines the environmental layers into a single suitability classification, employment and household disaggregation down to fine-grained spatial zones, aggregation, and a final validation and export stage. Every stage reads and writes through the same project geodatabase, so tools compose cleanly without any tool needing to know how another one is implemented internally.

Configuration-driven business rules everywhere. Classification logic, county quality tiers, and validation thresholds all live in version-controlled configuration tables rather than hardcoded in any individual tool. A correction to a business rule is a data edit, not a code change and redeploy, and every tool that depends on that rule reads the same source of truth.

Performance engineering where it actually matters. The stages that touch the largest volumes of data, spatial assignment of parcels to zones and the multi-layer terrain overlay, are built with the processing order and tooling chosen specifically for scale: joining small layers in first, running expensive geometric operations only where unavoidable, and parallelizing the single most expensive join across worker processes. That turned multi-hour operations into ones that finish in well under an hour.

Governance gates, not just automation. At several points the pipeline stops and requires a human decision before continuing, county-level data quality tiers must be reviewed and approved before classification runs, and any validation failure blocks a final export until either the underlying issue is fixed or an analyst explicitly acknowledges a warning. Automation handles the repetitive work, but consequential decisions stay with a person.

Defensive data engineering, built from real failures. More than one tool in this toolbox exists in its current form because an earlier version had a real, documented failure mode, a data source silently returning bad data behind a success status, or an overlay's base layer choice that could silently drop land from a total if a supporting layer had a gap. Each of those became a design principle applied consistently going forward: validate what a download actually contains, choose a foundation that structurally can't lose data, and check every output against an independent total before trusting it.

Audit logging as a first-class output, not an afterthought. Every tool in the toolbox writes a structured log covering what it read, what decisions it made, and what it produced, alongside the primary data output. Months later, tracing why a specific number in the final export looks the way it does is a matter of reading a log file, not reverse engineering the code.

Technical Highlights

- **Language/Platform:** Python, ArcGIS Pro (`arcpy`) Python Toolboxes, standalone Python scripts (including OGR/GEOS-based tools) for performance-critical stages
- **Scope:** Roughly twenty linked tools spanning configuration management, data acquisition, geometry cleaning, spatial joins, multi-layer overlay, classification,

disaggregation, aggregation, and validation

- **Data sources integrated:** County parcel records, USDA soil survey data, FEMA flood hazard layers, EPA water service boundaries, state employment establishment data, and Census housing statistics
- **Performance engineering:** Parallelized spatial joins using OGR/GEOS and a process pool, overlay operations ordered by feature count, in-memory intermediate processing to avoid unnecessary disk I/O
- **Data governance:** Config-driven business rules in version-controlled tables, county-level quality tiering, and analyst approval gates enforced directly in code
- **Data integrity:** Download and file validation beyond status codes alone, base-layer choices for overlays that guarantee full coverage, and reconciliation checks against independent totals
- **Reporting:** Structured, consistent audit logs from every tool, covering inputs, processing decisions, and output statistics for every run

Why It Matters

A statewide data system like this earns trust the same way any large piece of engineering does, not by being clever in any one place, but by being consistent, auditable, and resilient to the specific ways its inputs actually fail. Building it as a set of small, composable tools around shared configuration and shared governance gates, rather than one large script, means the system can absorb new data sources, new business rules, and new failure modes over its lifetime without needing to be rebuilt from scratch every time something changes.